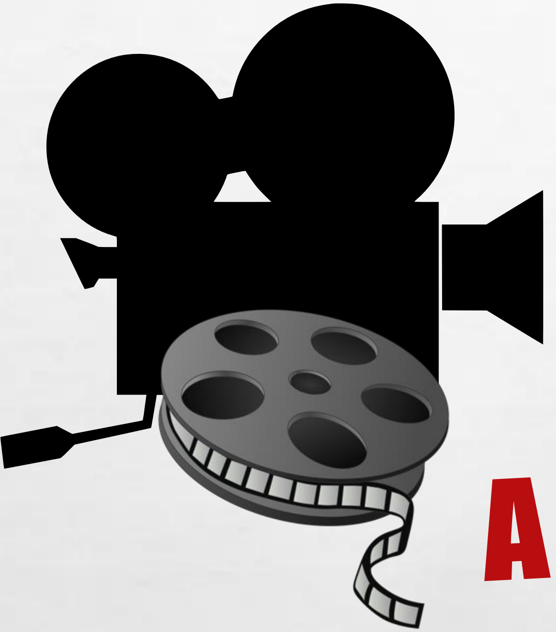


What to watch next…
A movie recommendation script

Salvatore Valeo

PROJECT OVERVIEW

- PROBLEM:
  TOO MANY MOVIES WITH TOO LITTLE TIME.
- SOLUTION:
  BUILD A RECOMMENDATION SYSTEM USING STRUCTURED METHODS, SUCH AS CRISP-DM
- RESEARCH QUESTION:
  HOW CAN USER BASED COLLABORATIVE FILTERING PROVIDE ACCURATE AND PERSONALIZED MOVIE SUGGESTIONS?

COLLABORATIVE FILTERING

* AN INFORMATION RETRIEVAL METHOD THAT RECOMMENDS ITEMS TO USERS BASED ON HOW OTHER USERS WITH SIMILAR PREFERENCES HAVE INTERACTED *

- COMPUTE COSINE-SIMILARITY:
  * MEASURES SIMILARITY BETWEEN TWO INDEPENDENT VECTORS
- GATHER RATINGS:
  PREDICT RATINGS FROM SIMILAR USERS
  RECOMMEND TOP-RATED UNSEEN MOVIES WITH SIMILAR ATTRIBUTES

DATASET OVERVIEW

- DATASET: MOVIE-LENS 100K (KAGGLE)
  100,000 RATINGS FROM 943 USERS ON 1,682 MOVIES
- FIELDS:
  USER_ID, MOVIE_ID, RATING, TIMESTAMP
  WITH INFORMATION ON THE MOVIE'S GENRE AND TITLE

index	user_id	movie_id	rating	timestamp
0	196	242	3	881250949
1	186	302	3	891717742
2	22	377	1	878887116
3	244	51	2	880606923
4	166	346	1	886397596

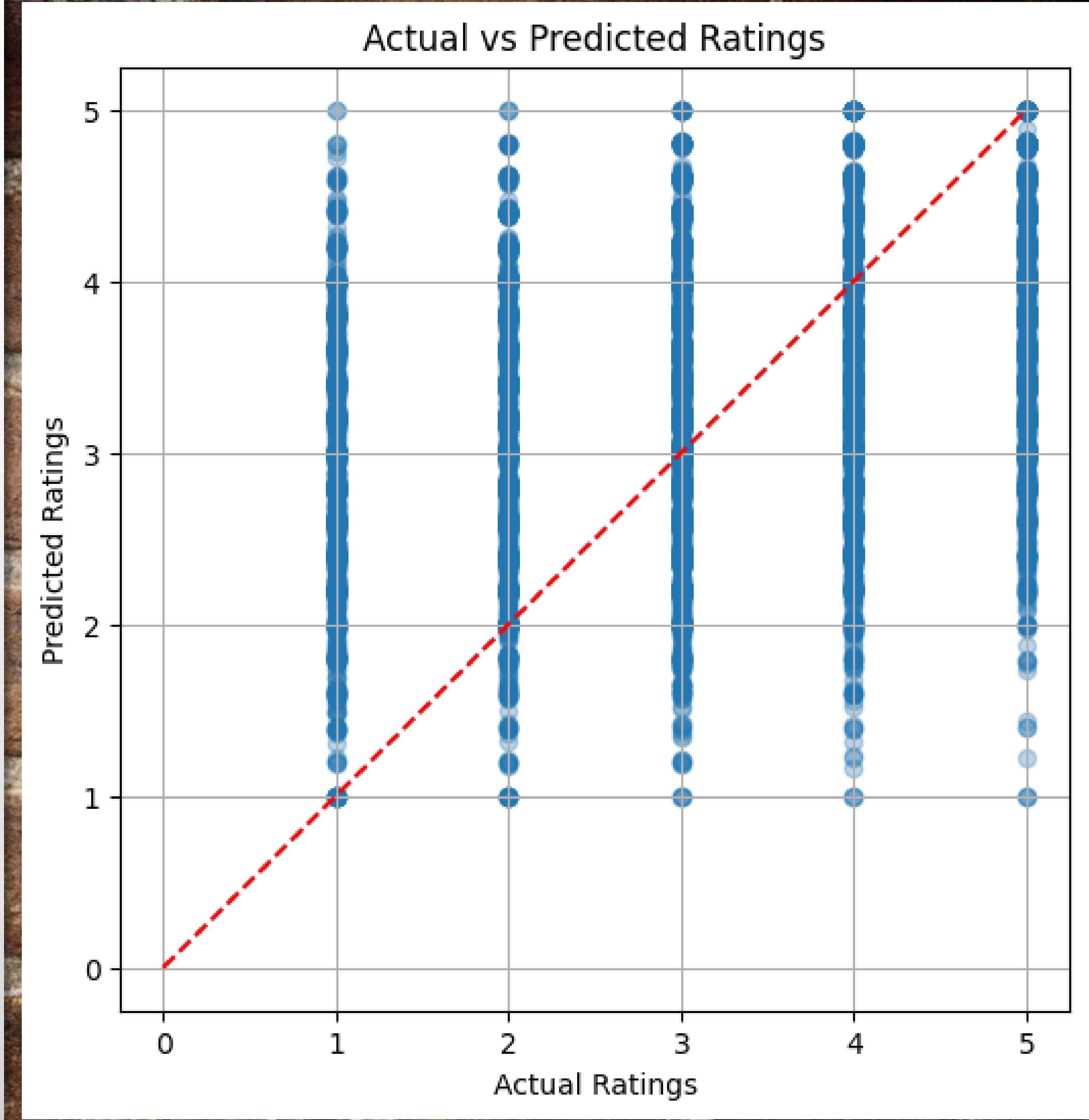
METHODOLOGY (CRISP-DM)

* UNDERSTAND THE NEED TO HELP USERS DISCOVER MOVIES THEY WILL ENJOY *

- DATA UNDERSTANDING:
  EXPLORED THE MOVIE-LENS DATASET, FOCUSING ON USER RATINGS AND CORRESPONDING FIELDS
- MODELING:
  DEVELOPED A RECOMMENDER SYSTEM USING SIMILARITY-BASED TECHNIQUES TO MATCH USERS WITH POTENTIAL MOVIES
- EVALUATION:
  MEASURED PREDICTION ACCURACY USING RMSE TO VISUALIZE ACTUAL VS PREDICTED RATINGS TO ASSESS PERFORMANCE.

EVALUATION METRICS

USED RMSE METRIC:
RMSE RATING: 1.0608



DEMONSTRATION

USER RATES A MOVIE 1-5 ‘STARS’

SCRIPT WILL RETURN A LIST OF MOVIES WITH SIMILAR ATTRIBUTES ALONG WITH A SIMILARITY RATING

```python
# Your ratings
your_ratings = {
    318: 5,   # Shawshank Redemption
    295: 5,   # Fight Club
    483: 3,   # Titanic
    296: 4,   # Pulp Fiction
    257: 5    # The Matrix
}

# Assign a new user ID
user_id = user_item_matrix.index.max() + 1

# Create a new user row with all 0s
user_row = pd.Series(0, index=user_item_matrix.columns)

# Insert your ratings
for movie_id, rating in your_ratings.items():
    user_row[movie_id] = rating

# Add to user-item matrix
user_item_matrix.loc[user_id] = user_row

# Recompute similarity matrix
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(user_similarity,
                                   index=user_item_matrix.index,
                                   columns=user_item_matrix.index)
```

```python
# Get top 5 recommendations for ME
your_recs = get_user_recommendations(user_id = user_id)

# Print out movie titles
print(f"\nTop 5 movie recommendations for ME:\n")
for movie_id, score in your_recs.items():
    title = movies[movies["movie_id"] == movie_id]["title"].values[0]
    print(f"{title} — Predicted Rating: {score:.2f}")
```

```
Top 5 movie recommendations for ME:

Fargo (1996) — Predicted Rating: 1.68
Seven (Se7en) (1995) — Predicted Rating: 1.68
Good Will Hunting (1997) — Predicted Rating: 1.67
Scream (1996) — Predicted Rating: 1.46
Glory (1989) — Predicted Rating: 1.45
```

LITERATURE REVIEW

- RECOMMENDATION SYSTEMS:
  RESNICK ET AL. (1994) – GROUPLENS (COLLABORATIVE FILTERING)
- MODEL TECHNIQUES:
  KORAN ET AL. (2009) – SVD RECOMMENDATION FROM THE NETFLIX TEAM
- DATASET:
  HARPER & KONSTAN (2016) – PAPER ON THE MOVIELENS DATASET
- WHY THESE THREE?
  THESE THREE LITERATURES PROVIDE THE MOST IMPORTANT TOPICS IN MY RESEARCH: COLLABORATIVE FILTERING, MODEL TECHNIQUES AND THE DATASET

CONCLUSION

- BUILT A WORKING, PERSONALIZED MOVIE RECOMMENDATION SYSTEM
- FOLLOWED CRISP-DM METHODOLOGY
- USED COSINE-SIMILARITY, KNN AND EVALUATION METRICS LIKE RMSE

THANK YOU!